SKILL CHAIN: CAMPUS PEER-TO-PEER SKILL EXCHANGE PLATFORM

Integrated OOPS and DBMS Project Report

Submitted by

VARSHA K – 250701807

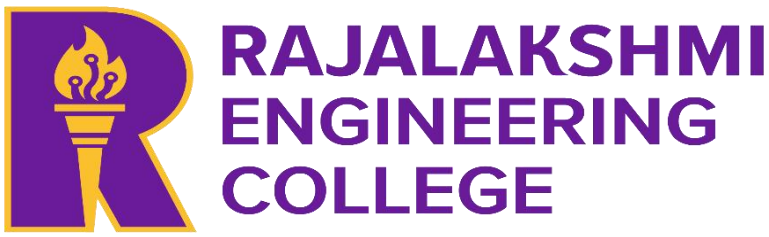
VANDHANA M – 250701804

VARSHINI S – 250701809

VARSSINI A – 250701811

of
BACHELOR OF ENGINEERING
in
Department of Computer Science and Engineering

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM
(An Autonomous Institution)



November 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled “SKILL CHAIN: CAMPUS PEER-TO-PEER SKILL EXCHANGE PLATFORM” is the bonafide work of “VARSHA K (250701807), VANDHANA M (250701804), VARSHINI S (250701809), VARSSINI A (250701811)” who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
Internal Faculty Name	Internal Faculty Name
Course Incharge	Course Incharge
Object Oriented Programming System	Database Management System
Department of Computer Science and Engineering	Department of Computer Science and Engineering
INTERNAL EXAMINER	**EXTERNAL EXAMINER**

ABSTRACT

In today's world, peer learning and knowledge sharing systems are very important and essential for academic ecosystems. This management system is an application-based system, to manage students and mark their skill portfolios. Every organisation whether government, private, or academic institutions uses an information system to store and coordinate data of their members. This project is useful for providing an easy and responsive graphical user interface.

The system uses the powerful relational database management system, data retrieval, and data manipulation. This project provides more ease for managing skill exchange data than manually maintaining and searching across disconnected chat groups. Hence it saves a lot of valuable student time. So, we can say that the project is useful for saving valuable time and reducing huge paperwork.

The main objective of this integrated project is to add, update, delete, view, search, and match student skill details. The Skill Chain: Campus Peer-to-Peer Skill Exchange Platform is a comprehensive solution designed to streamline and automate various aspects of peer knowledge exchange within a university campus. This project aims to develop a user-friendly and efficient desktop platform that facilitates student authentication, maintenance of teaching and learning skill portfolios, automated two-way reciprocal matchmaking, exchange request negotiations, active partnership tracking, and instant notifications.

Skill Chain captures and stores detailed student information including student names, institutional emails, salted password hashes, teaching proficiencies, and learning aspirations. This project helps in systematically organizing and querying peer data that needs to be accessed. The system was developed using foundational technologies such as MySQL database and Java (JDBC) with Java Swing GUI.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1	**INTRODUCTION**	
	• PROBLEM STATEMENT	5
	• PROJECT OVERVIEW	5
	• PURPOSE	5
	• SCOPE	5
2	**FUNCTIONALITIES PROVIDED**	**6**
3	**SYSTEM FLOW DIAGRAM**	
	• DATAFLOW DIAGRAM	8
	• USE CASE DIAGRAM	9
	• ENTITY RELATIONSHIP DIAGRAM	10
4	**MODULE DESCRIPTION**	**11**
5	**IMPLEMENTATION**	
	• JDBC CONNECTION CODE	12
	• MAIN LAUNCHER CODE	14
	• LOGIN AND REGISTRATION PAGE	16
	• DASHBOARD AND ACTIVE PARTNERSHIPS	21
	• ADD SKILL PAGE	26
	• VIEW AND SEARCH	30

SKILLS PAGE

• DELETE SKILL PAGE 34

• RECURSIVE CATEGORY TREE TAXONOMY 38

• CAMPUS STUDENT DIRECTORY PAGE 42

• RECIPROCAL MATCHING ENGINE 46

• LEARNING REQUESTS (ACCEPT / REJECT) 50

• NOTIFICATION CENTER 54

6 CONCLUSION 56

7 REFERENCES 57

- INTRODUCTION

PROBLEM STATEMENT

On university campuses, students frequently seek to acquire specialized technical, mathematical, and design skills outside their formal syllabus. However, discovering compatible classmates who possess the desired skills and are simultaneously eager to learn what the first student can teach is extremely challenging. Informal inquiries across campus social media groups are unorganized, unindexed, lack confidentiality, and fail to calculate two-way reciprocal compatibility. An automated, centralized student skill exchange platform is required to manage student profiles, catalog skills, compute mutual matches, and track exchange agreements securely.

PROJECT OVERVIEW

The Skill Chain: Campus Peer-to-Peer Skill Exchange Platform is a comprehensive solution designed to streamline and automate various aspects of peer knowledge exchange within an educational institution. This project aims to develop a user-friendly and efficient desktop platform that facilitates student authentication, maintenance of personal skill inventories (skills offered to teach and skills sought to learn), automated two-way reciprocal matchmaking, proposal requests negotiation, active partnership tracking, and real-time notification alerts.

Skill Chain captures and securely stores detailed student information including student names, verified university emails, cryptographically salted password hashes, and skill-type mappings. Through an automated relational Reciprocal Matching Engine, the platform pairs students when: Student A teaches what Student B wants to learn, and Student B teaches what Student A wants to learn. This project helps in systematically organizing and querying student data that needs to be accessed, ensuring zero monetary cost for peer learning.

PURPOSE

The system is aimed at total user-friendliness as well as efficient management of varied tasks:
• To centralize and organize student skill portfolios in a systematic manner and to facilitate easy access to accurate and up-to-date peer data, reducing manual errors and ensuring data integrity.
• To automate the identification of reciprocal exchange partners, eliminating days of manual inquiries.
• To provide a structured system for creating, sending, reviewing, accepting, and declining skill exchange proposals.
• To track active learning partnerships showing contact emails and mutual study obligations.
• To safeguard sensitive student credentials through cryptographically salted SHA-256 password hashing.
• To allow for potential expansion and inclusion of additional categories, ensuring the system can adapt to the evolving needs of the institution.

SCOPE

The system is aimed at total user-friendliness as well as efficient management of different tasks:
• Capturing and storing essential student information, including student names, university emails, password hashes, and registration timestamps.
• Providing a standardized academic catalog of skills categorized into programming, web development, data & AI, and digital design.
• Allowing students to tag skills as TEACH (skills offered) or LEARN (skills sought) with duplicate prevention.
• Enabling students to browse a campus-wide student directory displaying classmates' teaching and learning portfolios, equipped with direct 'Send Request ✉' action buttons.
• Executing automated dual-conditional matching queries using indexed MySQL joins.
• Managing exchange request lifecycles (PENDING, ACCEPTED, REJECTED) with notification indicators.
• Designing an intuitive, aesthetic desktop interface for students and administrators using Java Swing.

- FUNCTIONALITIES PROVIDED

FUNCTIONALITIES PROVIDED BY SKILL CHAIN PLATFORM:

• Simplifying and streamlining peer skill exchanges.
• Provide timely notifications and remainders for incoming requests and accepted swaps.
• Easy to understand.
• Easy to access.
• Easy to handle.
• Facilitate structured learning partnerships between university students.
• Automate reciprocal matchmaking calculations through relational multi-table self-joins.
• Centralize student skill portfolios with full CRUD (Create, Read, Update, Delete) capabilities.
• Prevent duplicate skill registrations through database unique constraints.
• Safeguard student passwords with 16-byte random salt and SHA-256 cryptographic hashing.

- SYSTEM FLOW DIAGRAM

System flow diagram helps in understanding the flow of information and processes within the system. The below displays such a representation for the project on Skill Chain: Campus Peer-to-Peer Skill Exchange Platform.

DATAFLOW DIAGRAM

A Data Flow Diagram (DFD) is a visual way to show how data moves through a system or process. In Skill Chain, student inputs flow through authentication, skill management, matching engine, and request negotiation processes connected to normalized MySQL tables.

USE CASE DIAGRAM

A use case diagram is a visual representation that depicts the interactions between actors (users) and a system, illustrating various ways the system can be utilized or the functionalities it offers.

Actor: Student Member Use Cases:
• Login to System – Allows an existing student to securely authenticate using registered university email and password.
• Register New Account – Allows a new student to create a profile with verified institutional email and confirmed password.
• View Dashboard & Metrics – Displays live metric counters for skills taught, skills learned, reciprocal matches, and exchange requests alongside active study partnerships.
• Add Skill – Allows a student to add a new skill to their profile designated as TEACH or LEARN.
• View / Search My Skills – Enables a student to view and filter their declared skill inventory.
• Remove Skill – Permits a student to delete an existing skill association from their portfolio.
• View Recursive Category Tree – Renders the academic taxonomy tree recursively.
• Browse Campus Student Directory – Displays all registered classmates, emails, and skill portfolios with direct proposal triggers.
• View Reciprocal Matches – Automatically calculates and displays students who share two-way teaching-learning synergy.
• Send Exchange Proposal – Dispatches a formal exchange invitation to a classmate.
• Accept / Reject Request – Allows receiving students to approve or decline incoming study proposals.
• View Notifications – Opens dynamic alerts regarding accepted swaps and incoming invitations.
• Exit System – Allows a student to log out or exit the desktop application safely.

ENTITY RELATIONSHIP DIAGRAM

An entity-relationship diagram (ERD) is a visual representation illustrating the relationships between different entities within a database system. Skill Chain models four core normalized relations: users, skills, user_skills, and learning_requests with cascading referential integrity constraints.

- MODULE DESCRIPTION

The different types of modules present in this project are:

LOGIN & REGISTRATION MODULE:
In this section, students can sign in or register new accounts with salted SHA-256 password protection.

STUDENT DASHBOARD & ACTIVE PARTNERSHIPS MODULE:

In this section, students can view dynamic metric cards and active ongoing study partnerships.

ADD SKILL MODULE:
In this section, students can add skills to teach or learn with uniqueness enforcement.

VIEW & SEARCH SKILLS MODULE:
In this section, students can inspect, filter, and search their skill inventory.

DELETE SKILL MODULE:
In this section, students can delete a skill from their portfolio with verification checks.

RECURSIVE CATEGORY TREE TAXONOMY MODULE:
In this section, students can view the academic taxonomy rendered via recursive N-ary tree traversal.

CAMPUS STUDENT DIRECTORY MODULE:
In this section, students can browse all registered classmates and send direct proposals.

RECIPROCAL MATCHING ENGINE MODULE:
In this section, an automated 4-table relational join computes mutual learning compatibility.

LEARNING REQUESTS MODULE:
In this section, students can review, accept, and reject exchange proposals.

NOTIFICATION CENTER MODULE:
In this section, dynamic alerts keep students updated on their invitations.

- IMPLEMENTATION

JDBC CONNECTION CODE :

```
package com.skillchain.util;

import java.io.File;
import java.io.FileInputStream;
import java.io.InputStream;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.Properties;

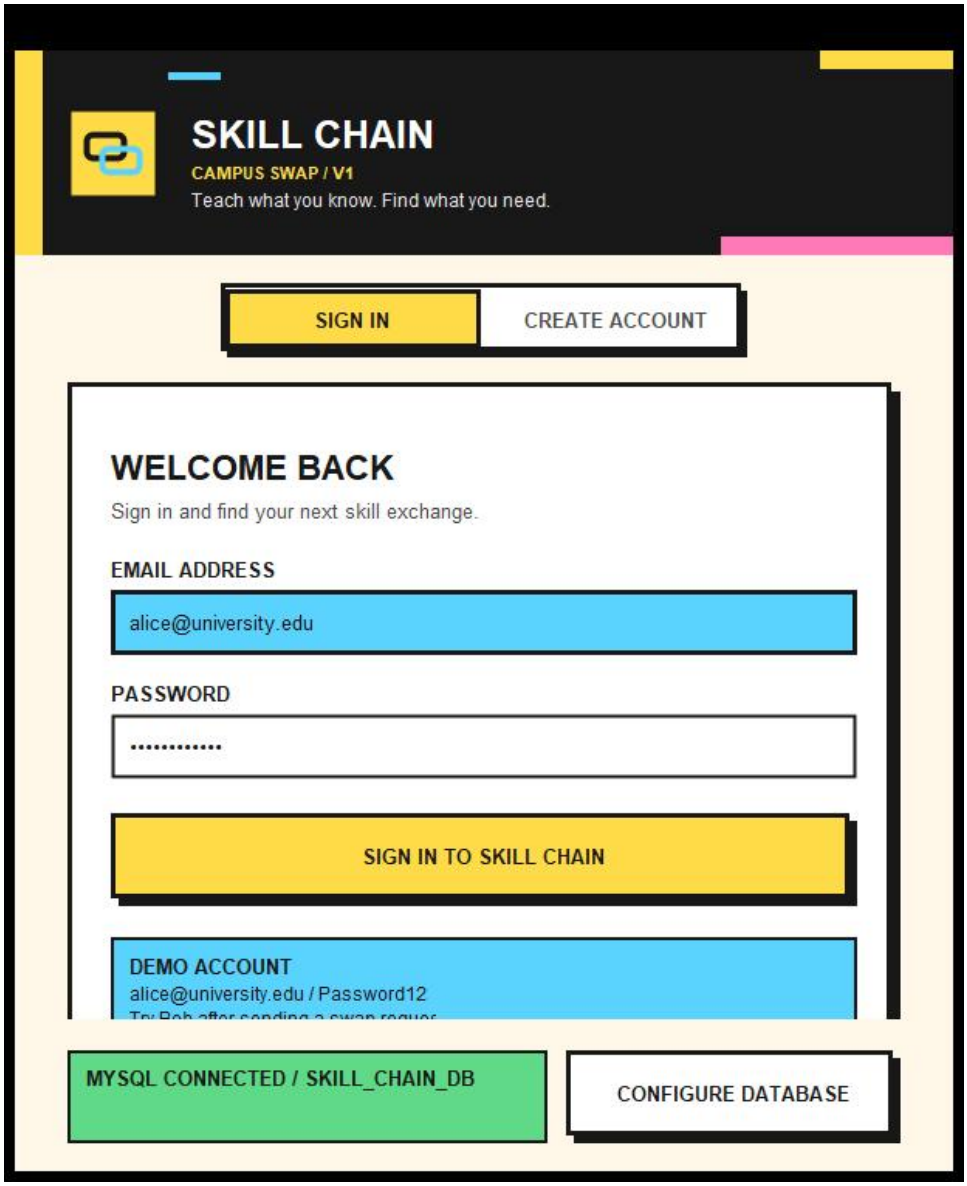
public final class DatabaseConnection {
    private static final String CONFIG_FILE_NAME = "config.properties";
    private static String dbUrl, dbUser, dbPassword;
    private static boolean initialized = false;
```

```
    private DatabaseConnection() {}

    private static synchronized void initialize() {
        if (initialized) return;
        Properties properties = new Properties();
        try {
            File ext = new File(CONFIG_FILE_NAME);
            if (ext.exists()) {
                try (InputStream fis = new FileInputStream(ext)) { properties.load(fis); }
            } else {
                try (InputStream cis =
DatabaseConnection.class.getClassLoader().getResourceAsStream(CONFIG_FILE_NAME)) {
                    if (cis != null) properties.load(cis);
                }
            }
            dbUrl = properties.getProperty("db.url");
            dbUser = properties.getProperty("db.user");
            dbPassword = properties.getProperty("db.password", "");
            Class.forName("com.mysql.cj.jdbc.Driver");
            initialized = true;
        } catch (Exception e) {
            throw new RuntimeException("DB Init Failed: " + e.getMessage(), e);
        }
    }

    public static Connection getConnection() throws SQLException {
        if (!initialized) initialize();
        return DriverManager.getConnection(dbUrl, dbUser, dbPassword);
    }
}
```

LOGIN PAGE :



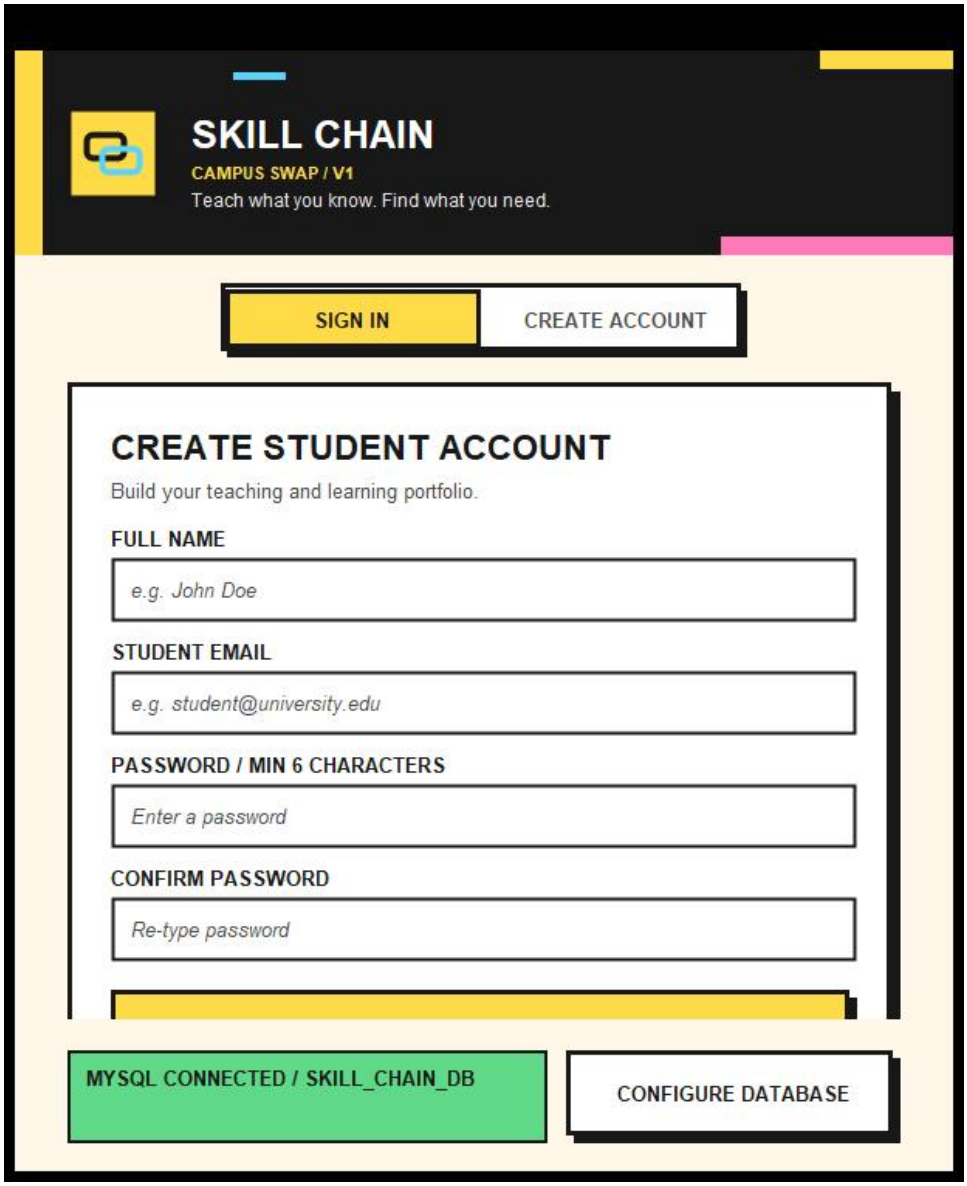
JAVA CODE:

```
// Login authentication verification in LoginRegisterFrame.java
User user = userService.login(email, password);
if (user != null) {
    JOptionPane.showMessageDialog(this, "Login Successful! Welcome, " + user.getName());
    new MainFrame(user, this).setVisible(true);
    setVisible(false);
}
```

JDBC QUERY CODE:

```
SELECT user_id, name, email, password_hash, salt, created_at FROM users WHERE LOWER(email)
= ?;
```

REGISTRATION PAGE :



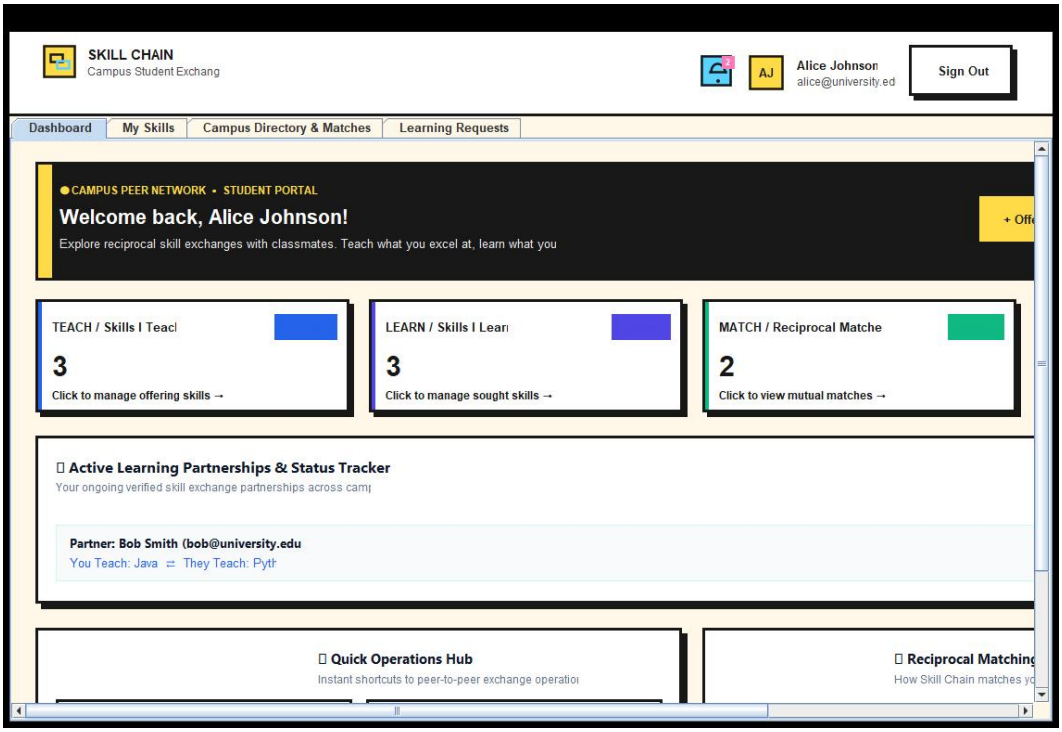
JAVA CODE:

```
// Registration logic with salted SHA-256 hash
User newUser = userService.register(name, email, password);
JOptionPane.showMessageDialog(this, "Student Account Created Successfully! Please Sign
In.");
```

JDBC INSERT CODE:

```
INSERT INTO users (name, email, password_hash, salt) VALUES (?, ?, ?, ?);
```

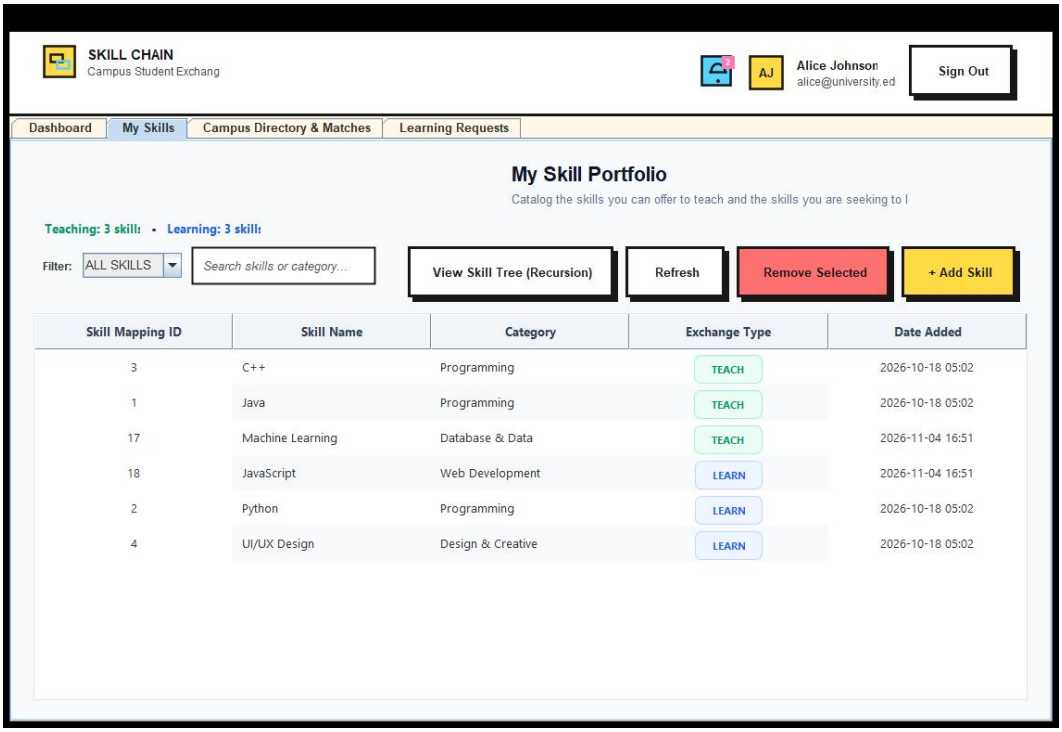
DASHBOARD & ACTIVE PARTNERSHIPS :



JDBC QUERY CODE (ACTIVE PARTNERSHIPS):

```
SELECT lr.request_id,
       CASE WHEN lr.sender_id = ? THEN u_recv.name ELSE u_send.name END AS partner_name,
       CASE WHEN lr.sender_id = ? THEN u_recv.email ELSE u_send.email END AS
partner_email,
       CASE WHEN lr.sender_id = ? THEN s_off.skill_name ELSE s_req.skill_name END AS
you_teach,
       CASE WHEN lr.sender_id = ? THEN s_req.skill_name ELSE s_off.skill_name END AS
partner_teaches,
       lr.status, lr.created_at
FROM learning_requests lr
JOIN users u_send ON lr.sender_id = u_send.user_id
JOIN users u_recv ON lr.receiver_id = u_recv.user_id
JOIN skills s_off ON lr.offered_skill_id = s_off.skill_id
JOIN skills s_req ON lr.requested_skill_id = s_req.skill_id
WHERE (lr.sender_id = ? OR lr.receiver_id = ?) AND lr.status = 'ACCEPTED'
ORDER BY lr.created_at DESC;
```

MY SKILLS PORTFOLIO (CRUD) :



JDBC INSERT (ADD SKILL):

```
INSERT INTO user_skills (user_id, skill_id, skill_type) VALUES (?, ?, ?);
```

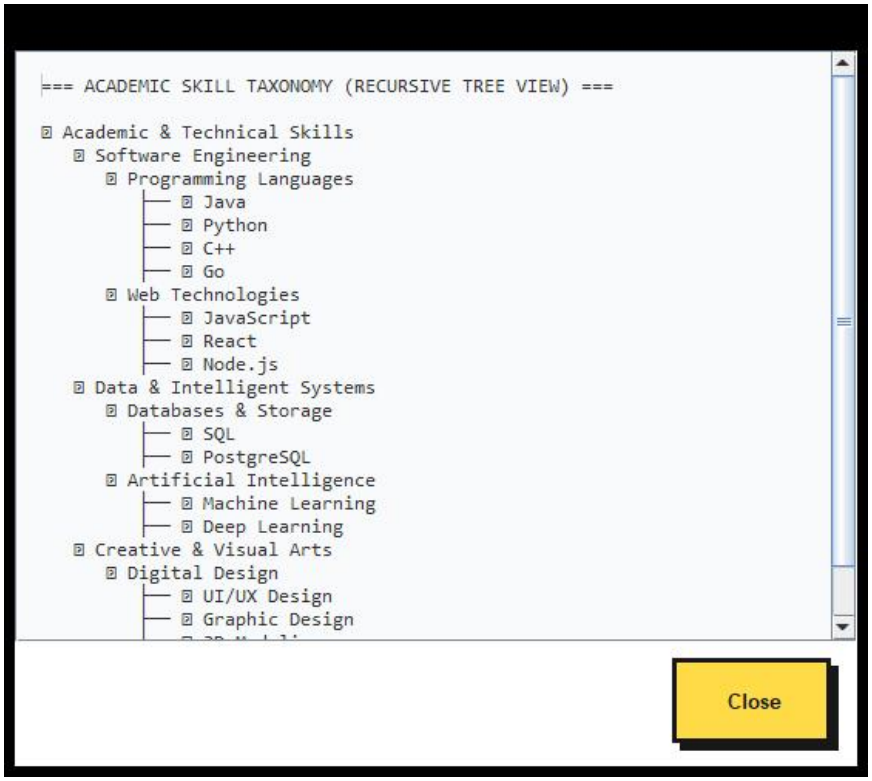
JDBC SELECT (VIEW SKILLS):

```
SELECT us.user_skill_id, us.user_id, us.skill_id, s.skill_name, s.category, us.skill_type,
us.created_at
FROM user_skills us
JOIN skills s ON us.skill_id = s.skill_id
WHERE us.user_id = ?
ORDER BY us.skill_type ASC, s.skill_name ASC;
```

JDBC DELETE (REMOVE SKILL):

```
DELETE FROM user_skills WHERE user_skill_id = ? AND user_id = ?;
```

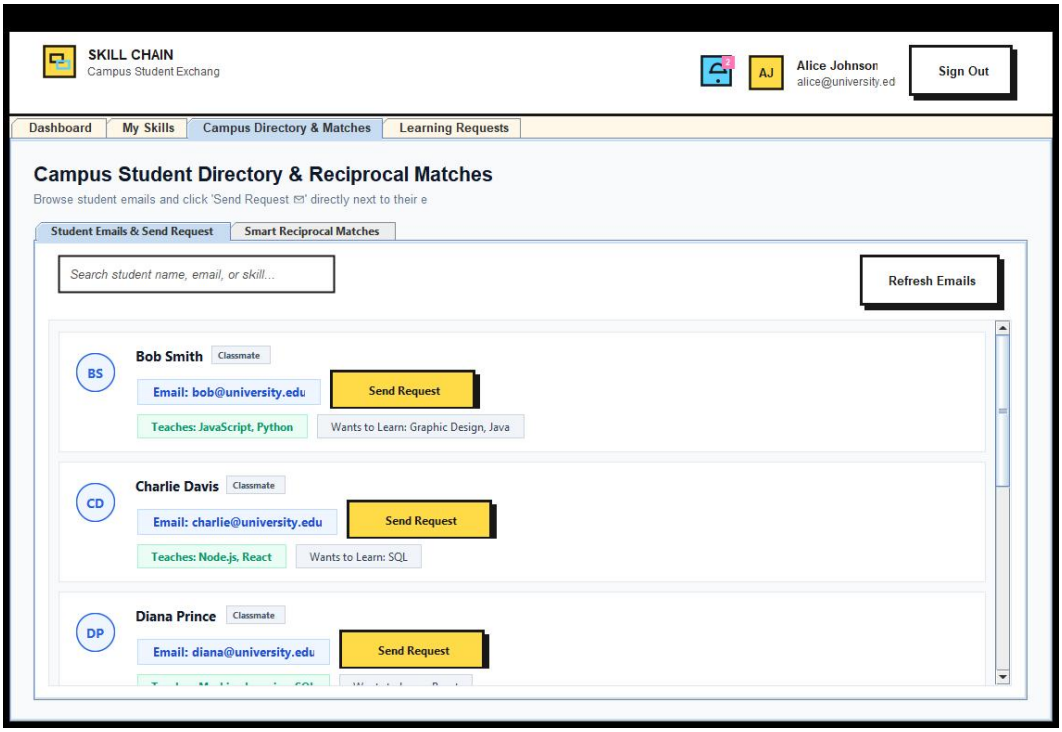
RECURSIVE CATEGORY TREE TAXONOMY :



JAVA RECURSIVE ALGORITHM:

```
public static void traverseTreeRecursive(SkillCategoryNode node, int depth, StringBuilder
buffer) {
    if (node == null) return; // Base Case
    String indent = "   ".repeat(depth);
    buffer.append(indent).append("🗁 ").append(node.getName()).append("\n");
    for (String skill : node.getSkills()) {
        buffer.append(indent).append("   ├── ⚡ ").append(skill).append("\n");
    }
    for (SkillCategoryNode child : node.getChildren()) {
        traverseTreeRecursive(child, depth + 1, buffer); // Recursive Step
    }
}
```

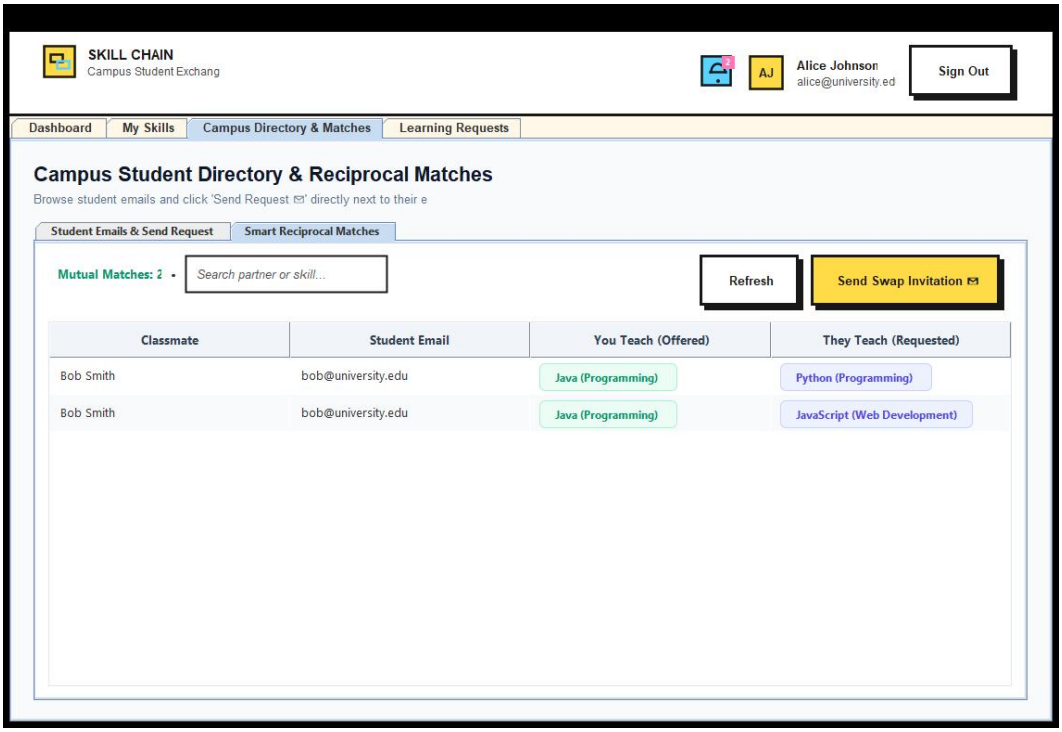
CAMPUS STUDENT DIRECTORY :



JDBC QUERY CODE:

```
SELECT u.user_id, u.name, u.email,
       COALESCE(GROUP_CONCAT(DISTINCT CASE WHEN us.skill_type = 'TEACH' THEN s.skill_name
END SEPARATOR ', '), 'None listed yet') AS teach_skills,
       COALESCE(GROUP_CONCAT(DISTINCT CASE WHEN us.skill_type = 'LEARN' THEN s.skill_name
END SEPARATOR ', '), 'None listed yet') AS learn_skills
FROM users u
LEFT JOIN user_skills us ON u.user_id = us.user_id
LEFT JOIN skills s ON us.skill_id = s.skill_id
WHERE u.user_id != ?
GROUP BY u.user_id, u.name, u.email
ORDER BY u.name ASC;
```

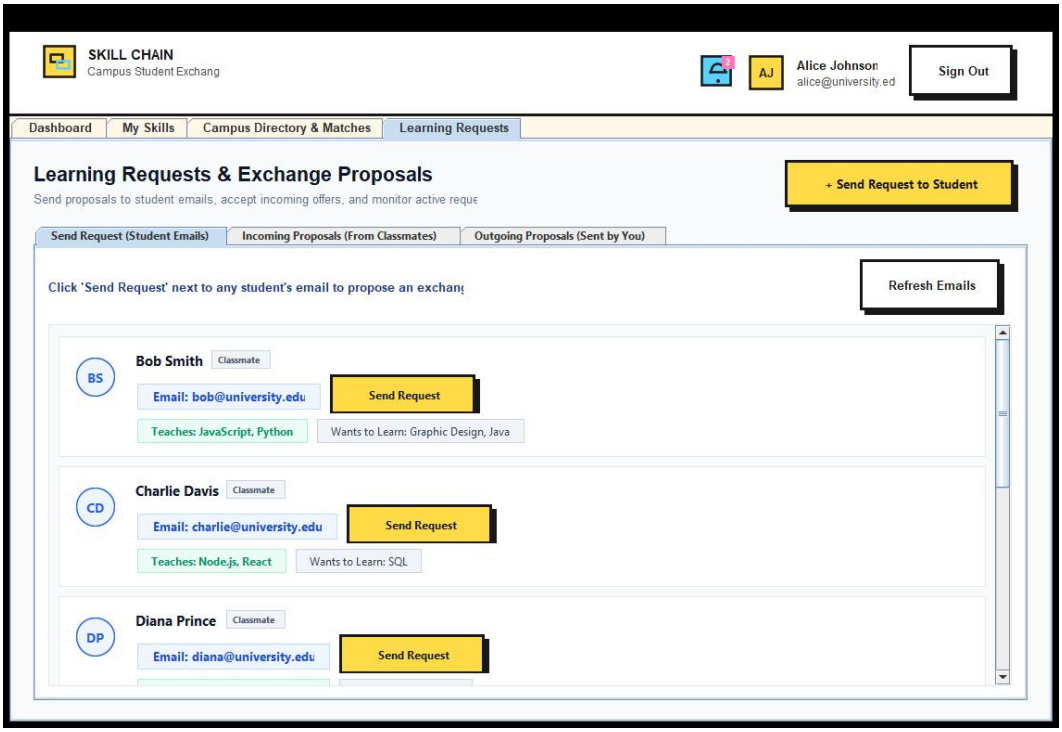
RECIPROCAL MATCHING ENGINE :

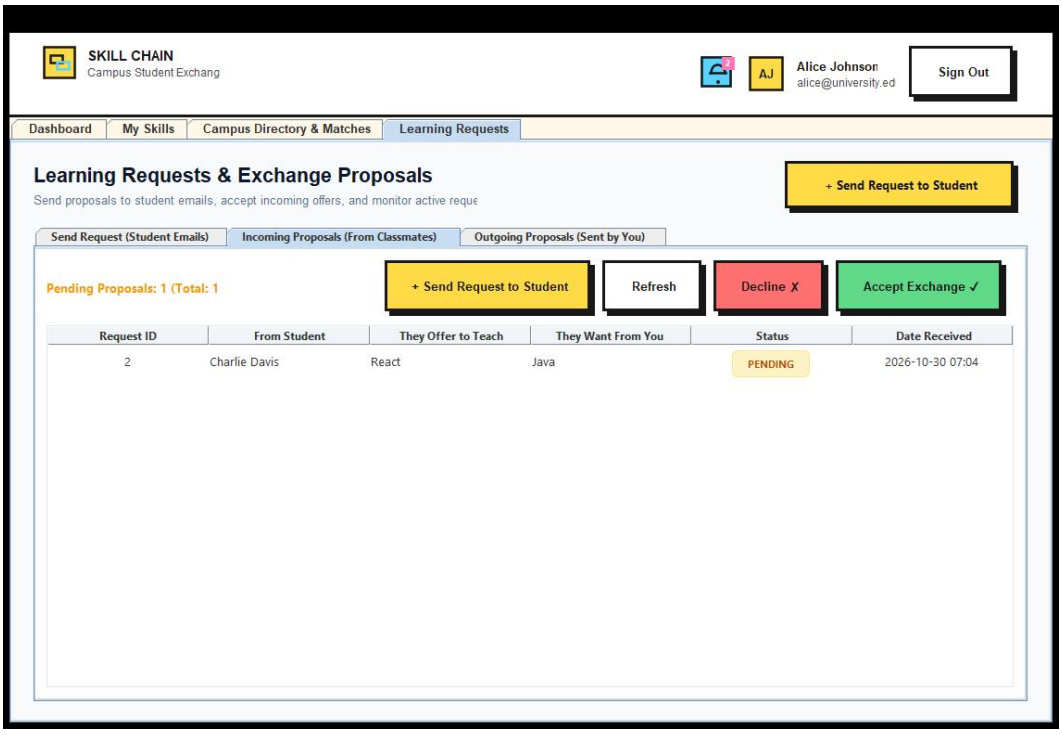


JDBC 4-TABLE RELATIONAL SELF-JOIN CODE:

```
SELECT u_partner.user_id AS partner_id,
       u_partner.name AS partner_name,
       u_partner.email AS partner_email,
       s_offer.skill_name AS user_offered_name,
       s_partner.skill_name AS partner_offered_name
FROM user_skills my_teach
JOIN user_skills partner_learn
    ON my_teach.skill_id = partner_learn.skill_id
   AND my_teach.skill_type = 'TEACH'
   AND partner_learn.skill_type = 'LEARN'
   AND partner_learn.user_id != my_teach.user_id
JOIN user_skills partner_teach
    ON partner_teach.user_id = partner_learn.user_id
   AND partner_teach.skill_type = 'TEACH'
JOIN user_skills my_learn
    ON my_learn.skill_id = partner_teach.skill_id
   AND my_learn.user_id = my_teach.user_id
   AND my_learn.skill_type = 'LEARN'
JOIN users u_partner ON u_partner.user_id = partner_learn.user_id
JOIN skills s_offer ON s_offer.skill_id = my_teach.skill_id
JOIN skills s_partner ON s_partner.skill_id = partner_teach.skill_id
WHERE my_teach.user_id = ?
ORDER BY u_partner.name ASC;
```

LEARNING REQUESTS (ACCEPT / REJECT) :

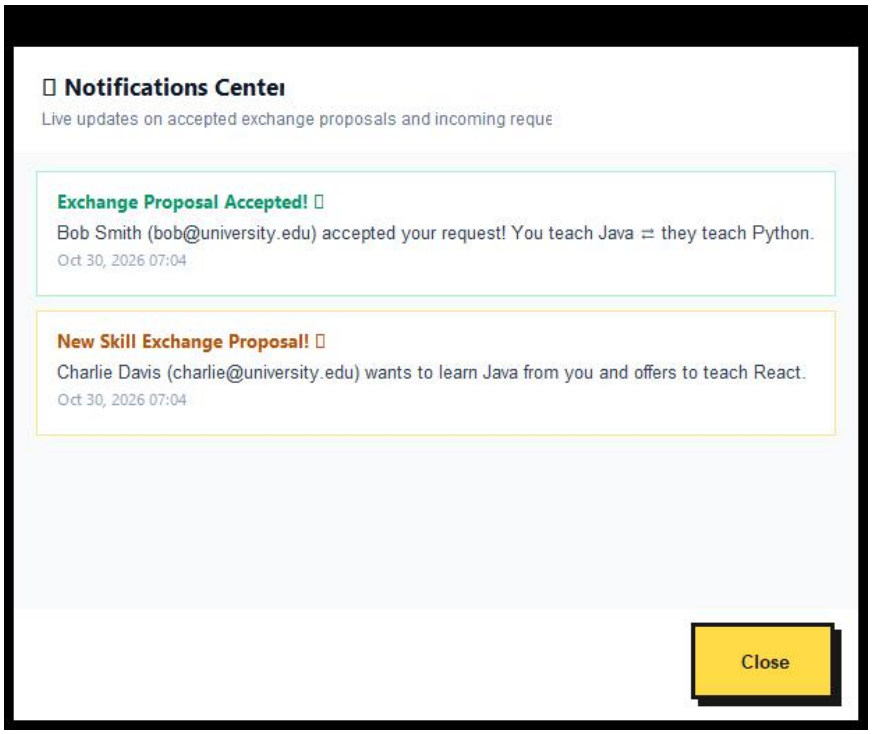




JDBC STATUS UPDATE CODE:

```
UPDATE learning_requests SET status = ? WHERE request_id = ?;
```

STUDENT NOTIFICATION CENTER :



- CONCLUSION

In conclusion, our student skill exchange database management system implemented in Java using JDBC successfully addresses the core functionalities of adding new skills, removing records, updating statuses, and efficiently retrieving data when needed. The utilization of JDBC allowed seamless interaction between the Java application and the MySQL database engine, ensuring robust data management and transaction security.

The project emphasized the importance of adopting core Object-Oriented Programming (OOPS) principles:

- Abstraction: separating interface contracts from database implementations.
- Inheritance: modeling domain users cleanly through base class reuse.
- Encapsulation: defensive input validation and getter/setter boundaries across all models.
- Polymorphism: compile-time method overloading and runtime dynamic method overriding.
- Data Structures & Recursion: traversing nested academic category trees using depth-first recursive algorithms.
- Application Security: cryptographically salted SHA-256 password hashing.

Additionally, the project's reliance on established database management principles, as outlined in key

texts, ensured a solid foundation for data integrity, 3rd Normal Form (3NF) relational design, referential cascading constraints, and multi-table join query optimization.

Through this project, we gained valuable insights into Java application development, database connectivity, and the significance of adhering to best practices in software design. The system provides a user-friendly interface for managing student skill portfolios, contributing to a more efficient and collaborative peer learning process on campus.

- REFERENCES

1. JAVA – The Complete Reference by ORACLE Press / Herbert Schildt.
2. Database System Concepts by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan (McGraw-Hill).
3. MySQL 8.0 Reference Manual – Oracle Corporation.
4. YouTube Playlist on Java Swing and JDBC Architecture: https://www.youtube.com/playlist?list=PLbKub4Jss9oUbTmeS9li_DoskYCiuvx1R